

TRABALHO 02

Análise Computacional de Problemas de Escalonamento

Machine Scheduling, Job Shop Scheduling e Flow Shop
Modelagem MILO em Julia com JuMP e HiGHS

Neivan Júnior Alves Costa
Aluno especial

Ambiente	Configuração
Julia	1.12.6
JuMP	1.30.1
HiGHS	1.23.0
Limite por MILO	10.0 segundos
Threads Julia	12
Workers paralelos	12
Threads por solver	1
Instâncias analisadas	39
Tempo total	45,22 segundos

Relatório técnico elaborado a partir dos materiais de aula e dos conjuntos de instâncias indicados na atividade.

Dashboard dos experimentos - visão geral

O painel executivo consolida as 39 instâncias oficiais analisadas e permite comparar indicadores, status e resultados de Machine Scheduling, Job Shop Scheduling e Flow Shop. A primeira visão reúne os filtros, indicadores e resultados detalhados sem interromper a imagem no início dos gráficos.

O dashboard interativo está disponível pelo link apresentado no README.md da pasta atividades/TRABALH002 no fork do autor. O relatório apresenta abaixo apenas o registro estático dos resultados.

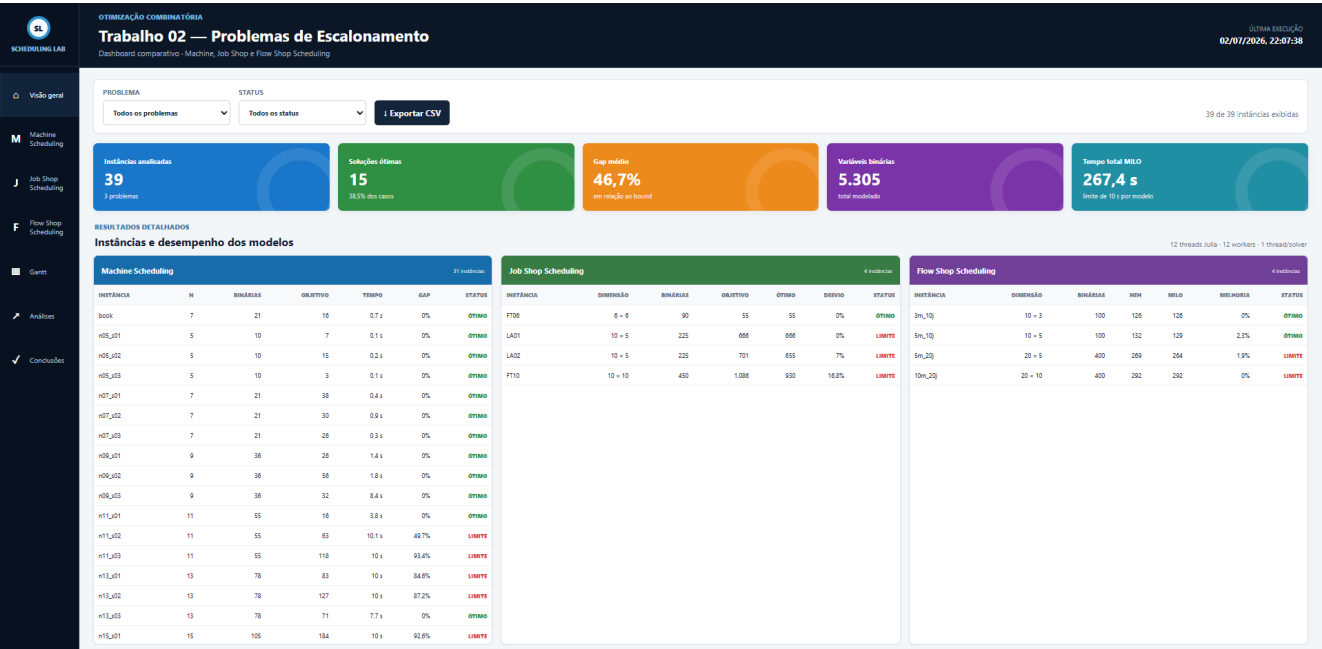


Figura 1 - Visão geral, indicadores e resultados detalhados do dashboard.

Os cartões superiores resumem volume, otimalidade, gap, variáveis binárias e tempo computacional. As tabelas preservam a rastreabilidade dos valores discutidos nas seções seguintes.

Análises e Gantt do dashboard

A segunda visão apresenta integralmente os gráficos comparativos, a distribuição de status, o Gantt ótimo da instância FT06 e os principais insights obtidos nos experimentos.

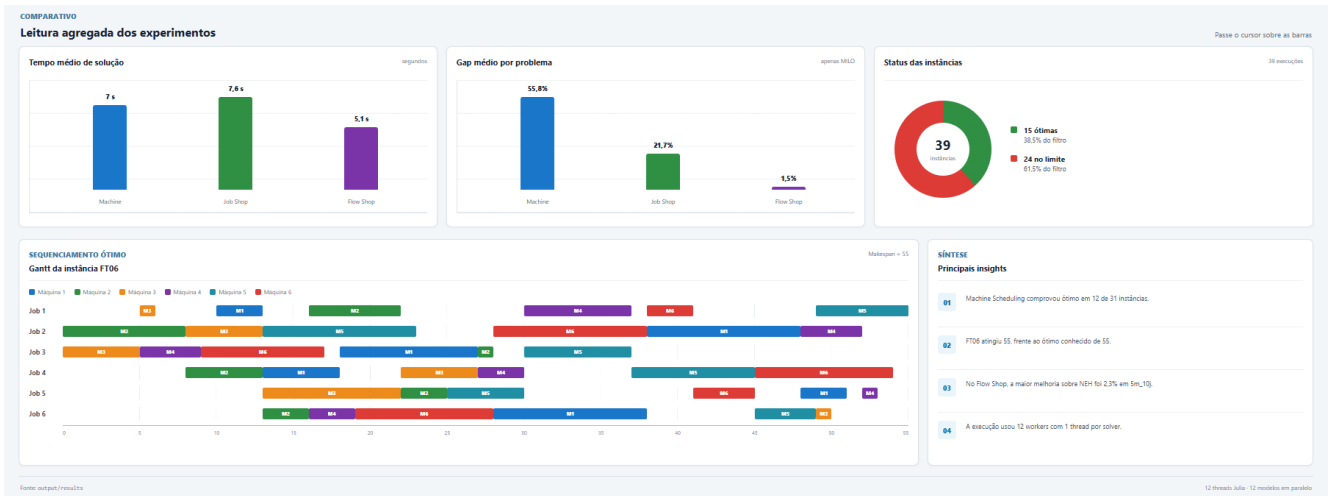


Figura 2 - Análises agregadas, Gantt da FT06 e síntese dos resultados.

Na versão web, os filtros por problema e status, os botões de navegação, as dicas dos gráficos e a exportação CSV permanecem funcionais. O acesso está indicado no README.md da pasta atividades/TRABALH002 no fork do autor.

Resumo

Este trabalho compara métodos heurísticos e modelos de programação inteira mista para três problemas clássicos de escalonamento. Em máquina única, foram comparadas as regras FIFO, EDD e SPT com uma formulação disjuntiva big-M que minimiza o atraso total. Para Job Shop, foi utilizado um modelo disjuntivo com precedências tecnológicas e minimização do makespan. Para Flow Shop de permutação, a heurística NEH foi comparada a uma formulação posicional sem big-M. Os resultados mostram que o método exato resolve rapidamente instâncias pequenas, mas passa a atingir o limite de tempo à medida que cresce o número de decisões binárias. As heurísticas têm custo praticamente desprezível e fornecem boas soluções, sobretudo no Flow Shop.

1. Objetivos e delineamento experimental

O objetivo é observar, de forma reprodutível, o compromisso entre qualidade da solução e esforço computacional. Cada modelo MILO recebeu limite de 10.0 segundos, gap alvo igual a zero e 1 thread por solver. Instâncias independentes foram distribuídas entre 12 workers Julia. Os tempos por instância correspondem apenas à chamada do otimizador, sem incluir a compilação inicial do Julia.

Machine Scheduling. Foram usadas as 31 instâncias JSON oficiais: o caso-base do MO-book e três sementes para cada tamanho n em $\{5, 7, 9, 11, 13, 15, 17, 19, 21, 24\}$. Tempos de processamento, liberações e folgas seguem o gerador fornecido no material AULA06.

Job Shop. Foram usadas as instâncias ft06, la01, la02 e ft10 do JSPLIB, com ótimos conhecidos iguais a 55, 666, 655 e 930.

Flow Shop. Foram usados quatro arquivos do conjunto akilelkamel/fssp-dataset: 3m_10j, 5m_10j, 5m_20j e 10m_20j. O NEH também foi usado como solução inicial do modelo posicional.

2. Formulações matemáticas

2.1 Máquina única com datas de liberação

Para cada tarefa j , sejam r_j a liberação, p_j o processamento, d_j a data de entrega, S_j o início, C_j a conclusão e T_j o atraso. Para cada par (i,j) , a variável binária y_{ij} escolhe a ordem relativa.

```
min sum_j T_j
S_j >= r_j; C_j = S_j + p_j; T_j >= C_j - d_j; T_j >= 0
C_i <= S_j + M(1-y_ij); C_j <= S_i + M y_ij
```

2.2 Job Shop

Cada operação (j,k) tem máquina e duração próprias. As restrições de precedência mantêm a ordem tecnológica de cada job, enquanto uma disjunção big-M impede sobreposição entre operações que usam a mesma máquina.

```
min Cmax
S_(j,k+1) >= S_(j,k) + p_(j,k)
Cmax >= S_(j,last) + p_(j,last)
Para operações a,b na mesma máquina: a antes de b ou b antes de a.
```

2.3 Flow Shop de permutação

A variável $x_{(j,k)}$ vale um quando o job j ocupa a posição k . As variáveis $C_{(k,i)}$ representam a conclusão da posição k na máquina i . A estrutura posicional dispensa big-M.

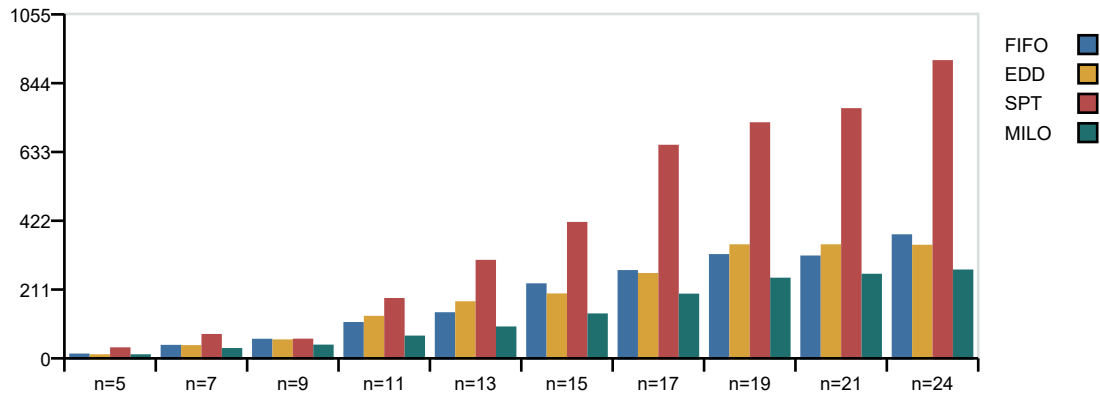
```
min C_(n,m)
sum_k x_(j,k) = 1; sum_j x_(j,k) = 1
C_(k,i) >= C_(k-1,i) + sum_j p_(j,i)x_(j,k)
C_(k,i) >= C_(k,i-1) + sum_j p_(j,i)x_(j,k)
```

3. Resultados - Machine Scheduling

A Tabela 1 apresenta a melhor regra de prioridade em cada instância e o resultado do MILO. Quando o status é TIME_LIMIT, o valor mostrado é a melhor solução incumbente encontrada, e não um ótimo comprovado.

Instância	n	Binárias	Melhor heur.	Obj. heur.	Obj. MILO	Tempo	Status	Gap
n05_s01	5	10	EDD	7	7	0,13 s	OPTIMAL	0%
n05_s02	5	10	EDD	15	15	0,15 s	OPTIMAL	0%
n05_s03	5	10	EDD	3	3	0,13 s	OPTIMAL	0%
book	7	21	EDD	27	16	0,68 s	OPTIMAL	0%
n07_s01	7	21	FIFO	42	38	0,39 s	OPTIMAL	0%
n07_s02	7	21	EDD	31	30	0,86 s	OPTIMAL	0%
n07_s03	7	21	EDD	30	26,00	0,30 s	OPTIMAL	0%
n09_s01	9	36	EDD	29	26	1,40 s	OPTIMAL	0%
n09_s02	9	36	SPT	64	56	1,79 s	OPTIMAL	0%
n09_s03	9	36	EDD	39	32	8,40 s	OPTIMAL	0%
n11_s01	11	55	EDD	43	16	3,78 s	OPTIMAL	0%
n11_s02	11	55	FIFO	98	63	10,14 s	TIME_LIMIT	49,67%
n11_s03	11	55	FIFO	178	118	10,01 s	TIME_LIMIT	93,41%
n13_s01	13	78	FIFO	116	83	10,01 s	TIME_LIMIT	84,57%
n13_s02	13	78	FIFO	214	127	10,03 s	TIME_LIMIT	87,24%
n13_s03	13	78	FIFO	82	71	7,73 s	OPTIMAL	0%
n15_s01	15	105	EDD	187	184	10,02 s	TIME_LIMIT	92,64%
n15_s02	15	105	EDD	236	155	10,01 s	TIME_LIMIT	96,23%
n15_s03	15	105	FIFO	139	62	10,02 s	TIME_LIMIT	59,31%
n17_s01	17	136	EDD	275	236	10,02 s	TIME_LIMIT	94,33%
n17_s02	17	136	FIFO	232	220	10,02 s	TIME_LIMIT	96,49%
n17_s03	17	136	FIFO	189	127	10,03 s	TIME_LIMIT	93,10%
n19_s01	19	171	FIFO	396	319	10,04 s	TIME_LIMIT	96,47%
n19_s02	19	171	FIFO	207	128	10,03 s	TIME_LIMIT	100%
n19_s03	19	171	EDD	313	283	10,02 s	TIME_LIMIT	96,93%
n21_s01	21	210	EDD	235	227	10,03 s	TIME_LIMIT	94,54%
n21_s02	21	210	FIFO	336	280	10,03 s	TIME_LIMIT	100%
n21_s03	21	210	FIFO	297	259	10,04 s	TIME_LIMIT	95,84%
n24_s01	24	276	EDD	241	150	10,05 s	TIME_LIMIT	100%
n24_s02	24	276	EDD	377	308	10,05 s	TIME_LIMIT	100%
n24_s03	24	276	EDD	415	347	10,05 s	TIME_LIMIT	98,46%

Tabela 1 - Desempenho em máquina única.

Atraso médio por tamanho e método (menor é melhor)

As regras de prioridade foram executadas em poucos microssegundos. O MILO obteve prova de otimalidade em 12 de 31 instâncias. Nos casos limitados, os gaps ficaram entre 49,67% e 100%. Gaps altos são compatíveis com a relaxação linear relativamente fraca de formulações disjuntivas big-M.

A dificuldade não depende apenas de n : a estrutura numérica da instância também influencia a busca. Os warm starts garantiram ao solver uma solução inicial competitiva e o MILO obteve melhoria máxima de 62,79% sobre a melhor regra simples.

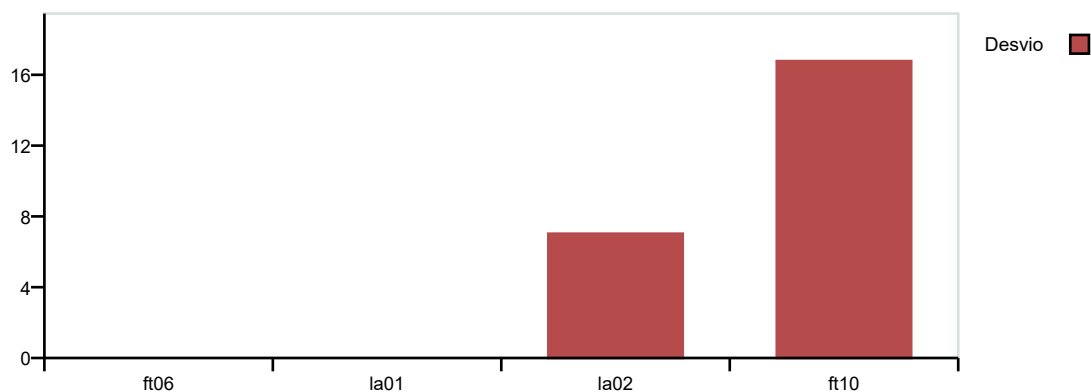
Entre as heurísticas, não houve uma regra universalmente dominante. As vitórias foram FIFO=13, EDD=17 e SPT=1. Portanto, regras simples são úteis como referências e warm starts, mas não substituem a otimização quando a qualidade da solução é crítica.

4. Resultados - Job Shop Scheduling

Instância	Dimensão	Binárias	Ótimo	Obtido	Desvio	Tempo	Status	Gap
ft06	6x6	90	55	55	0%	0,26 s	OPTIMAL	0%
la01	10x5	225	666	666	0%	10,08 s	TIME_LIMIT	22,64%
la02	10x5	225	655	701	7,02%	10,08 s	TIME_LIMIT	31,15%
ft10	10x10	450	930	1086	16,77%	10,08 s	TIME_LIMIT	33,00%

Tabela 2 - Resultados nas instâncias JSPLIB.

Desvio da melhor solução em relação ao ótimo conhecido (%)



A instância ft06, com 90 binárias, obteve 55 frente ao ótimo conhecido 55, em 0,26 s. 3 das 4 instâncias atingiram o limite. A comparação entre la01 e la02 confirma que tamanho sozinho não explica a dificuldade: a ordem das máquinas e os tempos alteram a geometria do modelo.

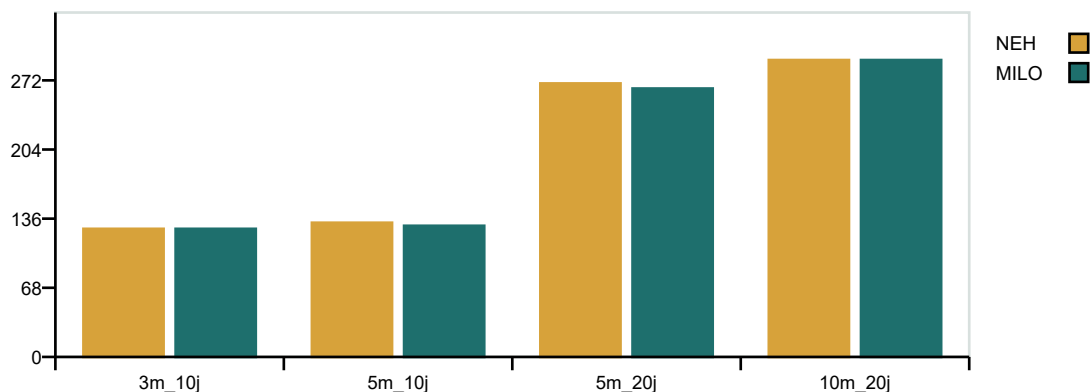
O maior desvio ocorreu em ft10: solução 1086, ou 16,77% acima do ótimo conhecido. Com limite de 10.0 segundos, o experimento evidencia a explosão combinatória nas instâncias maiores da formulação disjuntiva.

5. Resultados - Flow Shop

Instância	n	m	Binárias	NEH	MIL O	Melhoria	Tempo	Status	Gap
3m_10j	10	3	100	126	126	0%	0,02 s	OPTIMAL	0%
5m_10j	10	5	100	132	129	2,27%	0,40 s	OPTIMAL	0%
5m_20j	20	5	400	269	264	1,86%	10,03 s	TIME_LIMIT	1,89%
10m_20j	20	10	400	292	292	0%	10,03 s	TIME_LIMIT	4,15%

Tabela 3 - Comparação entre NEH e formulação posicional.

Makespan: NEH versus MILO (menor é melhor)



O NEH apresentou alta qualidade em todas as instâncias. O MILO comprovou ótimo em 2 de 4 casos. A maior melhoria ocorreu em 5m_10j: 129 contra 132 do NEH, ganho de 2,27%. O maior gap observado foi 4,15%.

Os gaps do Flow Shop foram muito menores que os observados nos modelos big-M. A formulação posicional possui n^2 binárias e aproveita melhor a estrutura do problema. Além disso, a inicialização com NEH garante desde o início uma solução viável de boa qualidade, permitindo que o solver concentre esforço na prova de otimalidade ou em melhorias marginais.

6. Discussão comparativa

Problema	Estrutura MILO	Crescimento das binárias	Comportamento observado
Máquina única	Disjunções big-M	$n(n-1)/2$	Ótimos pequenos; gaps altos em casos limitados
Job Shop	Precedências + disjunções big-M	Pares de operações por máquina	Maior dificuldade e desvios crescentes
Flow Shop	Atribuição posicional	n^2	Gaps baixos e forte benefício do NEH

A experiência confirma três pontos. Primeiro, o número de variáveis binárias é um indicador útil, porém incompleto: instâncias de igual tamanho podem ter dificuldades distintas. Segundo, formulações big-M simples são fáceis de implementar, mas podem gerar limitantes inferiores fracos e gaps elevados. Terceiro, heurísticas especializadas são especialmente valiosas como soluções iniciais. O NEH praticamente resolveu os casos de Flow Shop antes mesmo da otimização exata.

A comparação de tempos deve ser interpretada com cautela. Foi executada uma única rodada por instância, em um computador de uso geral, e as instâncias de máquina única foram geradas com sementes diferentes. Assim, os resultados caracterizam o comportamento dos métodos, mas não constituem um estudo estatístico de desempenho. A soma dos tempos individuais também difere do tempo de parede porque instâncias independentes foram executadas em paralelo.

7. Conclusões

Os modelos em JuMP com HiGHS reproduziram adequadamente os três problemas. Para instâncias pequenas, o MILO obteve e comprovou soluções ótimas. Para instâncias maiores, o limite de tempo expôs o crescimento combinatório, com efeito mais severo no Job Shop. As regras FIFO, EDD e SPT foram extremamente rápidas, mas sua qualidade variou entre instâncias. No Flow Shop, o NEH foi consistentemente competitivo e, combinado ao modelo posicional, forneceu o melhor equilíbrio entre qualidade e custo.

Como continuidade, recomenda-se testar limites de tempo maiores, múltiplas sementes para máquina única, formulações mais fortes, planos de corte e estratégias de melhoria local após o NEH. Também seria útil repetir cada configuração para estimar a variabilidade do tempo de solução.

Referências

- [1] Vinente, K. Optimization 2026 - materiais/AULA06. <https://github.com/kennyvinente/optimization2026/tree/main/materiais/AULA06>
- [2] MO-book. 3.5 Machine Scheduling. <https://mobook.github.io/MO-book/notebooks/03/05-machine-scheduling.html>
- [3] Tamy0612. JSPLIB: Benchmark instances for the Job Shop Scheduling Problem. <https://github.com/tamy0612/JSPLIB>
- [4] Elkamel, A.; Gharbi, A. Dataset for the Flow Shop Scheduling Problem, 2024. <https://github.com/akilelkaamel/fssp-dataset>
- [5] JuMP Developers. JuMP - Modeling language for mathematical optimization in Julia. <https://jump.dev>
- [6] HiGHS Development Team. HiGHS optimization software. <https://highs.dev>

Reprodutibilidade

O código completo está em `src/atividade02.jl`. Os dados selecionados ficam em `data/` e os CSVs produzidos em `output/results/`. A execução é feita com `julia --project=. --threads=auto src\atividade02.jl`. O dashboard lê os dados gerados automaticamente em `dashboard/data.js`.